eBPF'26 · 개별연구

UDP 소켓 드롭 비용

수신 처리율을 넘는 UDP 트래픽에서 소켓 레벨 드롭이 쓰는 CPU 측정

26.11.12 박수현

1. 배경: UDP 소켓 드롭

UDP 수신 경로 · 시간 순서



① TCP

흐름 제어. 수신 측이 느리면 송신 측이 속도를 낮춤

② UDP

흐름 제어 없음. 버퍼가 차도 계속 도착하고 소켓 큐 직전에서 드롭

드롭될 패킷도 큐 직전까지 softirq 처리 → 의미 없는 성능 저하

2. 실험 방법: 구성

물리 코어 3개 격리 · 송신 · 수신 · 모니터링에 하나씩 고정

송신

호스트

CPU 1

client

64B UDP 를 정해진 속도로 송신

수신

VM vCPU0

CPU 3

NIC 수신 처리 (softirq)

server: 패킷 처리

모니터링

VM vCPU1

CPU 4

bpftrace · 커널 카운터

1초마다 기록

수신 · 모니터링을 VM 에 두어 송수신이 실제 NIC 경로를 거치도록 구성

수신 코어가 드라이버부터 소켓 큐까지 softirq 전 과정을 거치도록

2. 실험 방법: 조건과 지표

udp-xdp-drop.sh · 속도마다 10초씩 측정

실험 조건

수신 속도 고정 · 송신 속도만 변경

수신	패킷당 20µs 처리 → 50,000 pps
송신 1배	50,000 pps
송신 2배	100,000 pps
송신 4배	200,000 pps

측정 지표

수신 코어(vCPU0) 기준 · 1초 단위

전체 패킷	`…/statistics/rx_packets`
처리 패킷	`Udp InDatagrams`
드롭 패킷	`Udp RcvbufErrors`
softirq	bpftrace 로 CPU 점유량 측정

수신 속도는 고정, 송신 속도만 1 · 2 · 4배로 변경

2. 실험 방법: softirq 측정

bpftrace · 수신 코어의 NET_RX softirq 실행 시간 누적

```
tracepoint:irq:softirq_entry /cpu == 0 && args→vec == 3/ { @s = nsecs; }
tracepoint:irq:softirq_exit /cpu == 0 && args→vec == 3 && @s/ { @t = @t + (nsecs - @s); @s = 0; }
interval:ms:10 { printf("%lld\n", @t); }
```

`softirq_entry`	수신 코어(cpu 0)의 NET_RX(vec 3) softirq 시작 시각 기록
`softirq_exit`	끝 시각 − 시작 시각을 @t 에 누적
`interval`	10ms 마다 누적값 출력, 모니터가 1초마다 읽음

softirq 점유량 = softirq 실행 시간 ÷ 전체 시간

2. 실험 방법: 예상 결과

송신 속도 1배 → 2배 → 4배

①

softirq 점유량 상승

드롭될 패킷의 수신 처리 증가

②

처리 패킷 감소

패킷 처리가 CPU 점유에서 밀림

송신이 빨라질수록 drop 연산에 의해 처리 패킷은 오히려 감소할 것으로 예상

3. 결과: 측정 출력

속도별 10초 · 표는 1~9초 평균 (10초 줄은 송신 종료 구간 포함)

```
psh@psh-MS-7E28:~/ebpf-net-experiments$ ./udp-xdp-drop.sh
== 50000 pps (1 배수) ==
sec  arrived_pps  goodput_pps  sock_drop_pps  softirq_%
1    49987        45847        4148           13.5
2    50001        45893        4105           13.2
3    50000        45859        4137           13.3
4    50000        45860        4143           13.3
5    49998        45870        4124           13.2
6    50003        45887        4117           13.3
7    50000        45875        4123           13.3
8    50000        45920        4073           13.5
9    49999        45981        4022           13.8
10   46228        42481        3716           12.7
== 100000 pps (2 배수) ==
sec  arrived_pps  goodput_pps  sock_drop_pps  softirq_%
1    99991        41502        58507          27.0
2    100001       41658        58339          26.6
3    99998        41634        58360          26.9
4    100002       41888        58128          26.9
5    99998        41855        58125          26.7
6    100003       41797        58204          27.0
7    100002       41871        58141          27.7
8    99996        41715        58266          27.8
9    99999        41825        58171          27.6
10   92560        38632        53818          25.0
== 200000 pps (4 배수) ==
sec  arrived_pps  goodput_pps  sock_drop_pps  softirq_%
1    199941       24855        175096         46.7
2    199990       24557        175435         46.7
3    200005       24680        175322         46.2
4    199992       24786        175189         46.5
5    200004       24408        175598         46.8
6    200005       24559        175458         46.3
7    199998       24219        175772         47.1
8    200005       24761        175251         46.7
9    199993       24638        175334         46.3
10   185021       23051        161682         43.2
```

송신 속도	softirq	처리 pps	드롭 pps
1배	13.4%	45,888	4,110
2배	27.2%	41,749	58,249
4배	46.5%	24,607	175,384

1배의 소량 드롭: recv() 시간이 더해져 실제 처리 속도가 약 46,000 pps 로 명목값보다 낮음

3. 결과: 예상과 비교

1배 → 4배 · 수신 코어(vCPU0)

① softirq 점유량 상승

13.4% → 46.5%

드롭 4,110 → 175,384 pps

② 처리 패킷 감소

45,888 → 24,607

처리 pps 46% 감소

예상한 두 가지 모두 확인

4. 결론

UDP 수신 용량 초과는 다른 프로세스에도 영향

---

1. 드롭될 패킷도 큐 직전까지 softirq 처리 → 수신 코어 점유 13.4% → 46.5%
2. 같은 코어의 처리 패킷 46% 감소 (45,888 → 24,607 pps)
3. XDP 단계에서 수신 현황을 보고 미리 드롭 → 성능 개선 기대